CLAUDE.md — Compilador C-

Contexto para agentes trabalhando neste repositório. Leia antes de editar qualquer coisa.

---

1. O que é

Compilador para **C-** (subconjunto de C do livro do Louden) escrito em C com flex + bison. Não gera código para uma máquina genérica: gera **binário de 32 bits para um processador MIPS-like específico**, implementado em Verilog no repositório irmão `../ProcessadorMips` (Quartus/FPGA).

Isso significa que `binario.c` **e** `ProcessadorMips/Controle.v` **são um contrato**. Mudar a tabela de opcodes de um lado sem o outro produz binário que roda errado em silêncio. Ver §5.

Projeto acadêmico. Comentários, nomes de função e mensagens de erro são em **português**.

---

2. Build & execução

```
make all                    # flex → bison → gcc -c *.c → cmenos
./cmenos entradas/gcd.txt   # exige exatamente 1 argumento
```

Saídas são gravadas em caminhos **fixos e relativos ao diretório atual** — rode sempre a partir de `Compiladores/`, senão os `fopen` falham silenciosamente (não há checagem de `NULL` na maioria):

Arquivo	Fase
`saidas/arvore.txt`	AST
`saidas/tabelaSimbolos.txt`	tabela de símbolos
`saidas/intermediario.txt`	quádruplas
`saidas/assembly.txt`	assembly
`saidas/binario.txt`	binário de 32 bits (carga da ROM)

Outros alvos: `make clean` (mantém `saidas/`), `make clean-all` (**apaga `saidas/`, que é versionado como exemplo**), `make rebuild`.

> **Nesta máquina Windows não há gcc, flex, bison nem make.** Use WSL, ou MSYS2 com `pacman -S gcc flex bison make`. Sem toolchain, só é possível análise estática.

`binarios/*.bin` guarda os binários de referência já validados — use para testes de regressão.

---

3. Pipeline: onde mexer

`main.c:52-62` encadeia as seis fases. Cada uma tem exatamente um ponto de entrada:

#	Fase	Arquivo	Entrada	Produz
1	Léxico	`lex.l`	`getToken()`	tokens
2	Sintático	`parser.y`	`parse()` → `TreeNode*`	AST
3	Semântico	`semantic.c`	`buildSymtab(raiz)`	tabela de símbolos (`symtab.c`)
4	Intermediário	`intermediario.c`	`gerarIntermediario(raiz)` → `Quad*`	quádruplas
5	Assembly	`assembler.c`	`geraAssembly(quads)` → `NoInstrucao*`	lista de instruções
6	Binário	`binario.c`	`geraBinario(inst)`	texto de 32 bits/linha

`util.c` tem `newStmtNode`/`newExpNode`/`copyString`/`insereEscopo`/`printTree`. `pilha.c` é uma pilha de strings usada só para empilhar parâmetros de chamada no assembler.

`noFlex.y` (724 linhas) é código morto — front-end alternativo com lexer manual em trie, não referenciado pelo `Makefile`. Ignore.

---

4. Estruturas centrais

`TreeNode` (`globals.h:28`) — AST com **3 filhos + irmão**. Listas (declarações, statements, parâmetros, argumentos) são encadeadas por `irmao`, não por filho. `escopo` é uma **string** (`"global"` ou o nome da função), atribuída em massa por `insereEscopo()` quando a função é reduzida em `parser.y:90-91`. Não há escopo de bloco: `int t;` dentro de um `while` recebe o escopo da função.

`Quad` (`intermediario.h:19`) — lista ligada de `(op, arg1, arg2, out)`. Cada `arg` é `{Vazio, Const, String}` (`tipoArg`). `String` guarda ou um nome de registrador (`"$t3"`, `"$rf"`), ou um nome de variável, ou um escopo, ou um label — dependendo da operação. **Sempre cheque `arg.type` antes de ler `conteudo.nome`.**

`NoInstrucao` (`assembler.h:62`) — instrução de saída. O campo `tipo` decide quais outros campos são válidos: `0`=J (usa `endereco`), `1`=R (usa `reg1/reg2/reg3`), `2`=I (usa `reg1/reg2/imediato`), `3`=IO (usa `reg1`), `4`=LCD (usa `imediato`).

`escopos` / `variavel` (`assembler.h:88-100`) — mapa de memória construído durante o assembly. `posicaoMemEscopo` é o deslocamento relativo a `$fp` (locais) ou ao topo (globais).

Estado global relevante: `intermediario.c` usa `arg aux` como "valor corrente" (o resultado de cada subexpressão) e `int *regs` como bitmap de 32 registradores. `assembler.c` usa `memLocG` (memória global gasta) e `numLinhas` (contador de instruções, base 1).

---

5. Contrato com o hardware (`../ProcessadorMips`)

Limites físicos — não há verificação de nenhum deles no compilador

Recurso	Limite	Fonte
ROM (instruções)	**256**	`ROMSinglePort.v:5` (`ADDR_WIDTH=8`)
RAM (dados)	**64 words**	`RAMDualPortDualClock.v:5` (`ADDR_WIDTH=6`)

`sort.bin` já ocupa 153 das 256 instruções. Na RAM, **globais crescem do endereço 63 para baixo** (`63-memLocG`) e a **pilha cresce de 0 para cima** — elas colidem em silêncio.

Registradores (`assembler.h:55-60`)

`$zero`=0 · `$t0..$t25`=**1..26** · `$aux`=27 · `$rf`=28 · `$fp`=29 · `$sp`=30 · `$ra`=31

⚠️ **Só `$t0..$t25` são alocáveis.** `solicitaReg()` gera nomes até `$t31`, mas `pegaRegistrador()` mapeia `$t26`→`$aux`, `$t28`→`$fp`, `$t29`→`$sp`, `$t30`→`$ra` e `$t27`/`$t31`→`99` (índice inválido). Ver B-02 em `BUGS.md`.

`$aux` é scratch do assembler (materializa imediatos). `$rf` carrega o valor de retorno.

Convenção de chamada

- Prólogo (`fun`, só para funções ≠ `main`): `SW $sp,$ra,0` · `ADDI $sp,$sp,1` · `SW $sp,$fp,0` · `ADDI $sp,$sp,1` · `SUBI $sp,$fp,2`
- Parâmetros: o chamador faz `SW $sp, <valor>, 2+i` **antes** do `JAL`
- Epílogo (`endF`): `SUBI $sp,$sp,<mem do escopo>` · `LW $fp,$ra,0` · `LW $fp,$fp,1` · `JR $ra`
- Retorno: `move` para `$rf`, depois `jump END_<função>`
- `main` **não tem prólogo nem epílogo** e **nenhum `HALT` é emitido** (B-06)

Tabela de opcodes: compilador × hardware

Comparação de `binario.c` com `Controle.v:27-793`. O hardware decodifica `000000`–`100101`; qualquer outro opcode cai no `default`, que deixa **todos os sinais de controle em `X`**.

Corretos e efetivamente emitidos:

Mnemônico	Opcode	Hardware
ADD / SUBB / MULT / DIVV	`000000` `000001` `000010` `000011`	ADD / SUB / MULT / DIV
SLT	`001000`	SLT
SLET	`011000`	ULA `1001` = `<=`
SET	`100011`	SET
SLETI	`100101`	SLETI
DIVI	`100100`	DIVI
ADDI / SUBI	`001111` `010000`	ADDI / SUBI
BEQ	`010011`	BEQ
LW / SW	`011001` `011010`	LW / SW
JUMP / JAL / JR	`011100` `011101` `001100`	J / JAL / JR

Mnemônico	Opcode	Hardware
INPUT / OUTPUT	`100010` `100000`	INPUT / OUTPUTREG

Divergentes — o binário roda outra instrução:

Mnemônico	Opcode emitido	O que o hardware executa	Emitido hoje?
MULTI	`000101`	**OR**	**sim** (`x * constante`)
SETLCD	`100110`	**não decodificado** (`default`, sinais em `X`)	**sim** (`entradas/so.txt`)
SGT / SGTI	`""` (vazio)	instrução truncada	não (B-01)
SGET / ANDI / SGETI	`001010`	MFLO	não
SDT / ORI / SDTI	`001100`	JR	não
SLL / MODI	`001101`	JALR	não
SRL / BLTZ	`001110`	LA	não
MOD / BGTZ	`001111`	ADDI	não
SLTI	`000111`	SRL	não
BEQZ	`010111`	BLT	não
BNE	`011000`	SLET	não
LUI	`010101`	BGT	não
SETI	`001011`	MOVE	não

Ao adicionar suporte a uma instrução nova, **confira o opcode contra `Controle.v` antes** — a maior parte da tabela em `binario.c` nunca foi validada.

Endereçamento de labels

`criaLabel()` grava `numLinhas`, que é o índice **base 1** da próxima instrução. `binario.c` grava `pegaLinhaLabel(...) - 1` para converter ao PC base 0 — em `JUMP`, `JAL` e `BEQ`. Se você criar um novo tipo de instrução com destino, **não esqueça o `-1`**.

---

6. Estado real

Validados end-to-end (`NOTES.txt` + `binarios/`): `sort` `gcd` `areaTriangulo` `volume` `fibonacciI` `factorial` `primo` `inverterArray` `buscaArray` `input`.

Não funciona: `maiorArray.txt` — é o único teste que usa `>`, e `>`/`>=`/`!=` não geram assembly (B-01). Não é coincidência que seja o único fora da lista.

Frágil: recursão (`gcd`) funciona por sorte — os quads `empilha`/`desempilha` são gerados mas o assembler os ignora (corpos comentados), então registradores vivos não são salvos na chamada (B-11).

Sem hardware: `setLcd` foi adicionado no último commit do compilador ("display lcd"), mas `../ProcessadorMips` não tem nenhuma referência a LCD.

Não há testes automatizados. A suíte de fato são os `.bin` em `binarios/`.

---

7. Armadilhas ao editar

1. **`malloc(strlen(s) * sizeof(char))` seguido de `strcpy` está em ~30 lugares** e falta o `+1` do terminador. Ao copiar código existente você propaga o bug. Sempre `strlen(s) + 1`.
2. **Buffers são usados sem inicializar.** `cria_label()` faz `strcat` em memória fresca de `malloc`; `binario.c` idem no ramo `INPUT`. Hoje passa porque o glibc devolve página zerada. As 652 linhas de `binarios/*.bin` têm exatamente 32 caracteres — o binário está certo **por sorte**, não por construção. Qualquer mudança no padrão de alocação pode quebrar tudo de uma vez.
3. **`insereQuad()` libera registradores como efeito colateral** (`intermediario.c:108-123`), com base no opcode. Se adicionar uma operação nova que consome registradores, ela precisa entrar nessa lista — senão os registradores vazam e você esgota os 26 disponíveis.
4. **Erros semânticos não abortam a compilação.** `insertNode()` só faz `printf`; a flag `error` (`main.c:17`) nunca é lida; `main()` sempre retorna `0`. Um programa com variável não declarada segue direto para a geração de código.
5. **Sempre teste `arg.type` antes de ler `conteudo.nome`.** Vários pontos do assembler assumem `String` e desreferenciam um `int` como ponteiro quando recebem `Const` (B-05).
6. **O `Makefile` esconde todos os avisos** (`gcc -c *.c`, sem `-Wall`). Ao mexer, compile com `gcc -c -Wall -Wextra *.c` para ver o que o build normal engole.
7. **`YYSTYPE` é `TreeNode*`** (`parser.y:15`) e as ações usam `tokenString` (global do lexer) para pegar o texto do token. Isso só funciona porque bison faz *default reduction* sem lookahead nos estados de `id : ID` e `fator : NUM`. Adicionar produções que criem conflito nesses estados quebra os nomes de identificadores de forma não óbvia.

---

8. Bugs conhecidos

Ver **`BUGS.md`** — 32 itens priorizados por severidade, com `arquivo:linha`, reprodução e correção sugerida. Os seis críticos: B-01 (`>`/`>=`/`!=` sem assembly), B-02 (registradores aliasando `$sp`/`$fp`/`$ra`), B-03 (`MULTI` executa `OR`), B-04 (segfault em vetor local), B-05 (segfault em condição constante), B-06 (`HALT` nunca emitido).